

AZTHREUS SYSTEMS: COMPANY OVERVIEW & ENGINEERING STANDARDS

Document ID: AZ-CONSOLIDATED-01 | Classification: Internal Owners: CEO, CTO, Engineering Lead
| Review Cycle: Annual/Bi-Annual

1. CORPORATE IDENTITY

1.1 Mission, Vision & Values

Mission: Architect fault-tolerant software, quantum computing frameworks, and delay-tolerant networks enabling human expansion into deep space. Engineer resilience, ensure survival of human knowledge, and bridge vast distances through unparalleled technological precision. **Vision:** Serve as foundational digital infrastructure for a multi-planetary civilization — the universal standard for interstellar software engineering and perpetual archive of digital human heritage over the next 150 years. **Core Values:** Resilience (survive extreme environments and generational timeframes), Precision (mathematically verifiable, fault-tolerant code), Stewardship (long-term protection of human data and heritage), Exploration (advancing quantum and interstellar algorithms).

1.2 Organizational Structure (Flat Technical Hierarchy)

Executive Board → CEO → CTO (Systems Architecture, Quantum Engineering, Deep Space Comms, R&D)
→ CISO (Info Security, CIRT)
→ COO (Mission Operations, Infrastructure, Supply Chain)
→ HR Director, Quality Manager, Legal & Compliance Officer, Finance Officer

Communication Norms: Internal messaging platform for daily communication; email reserved for formal/external. Teams hold 15-min daily standups + weekly planning. All architectural-impact decisions recorded as ADRs. Safety/escalation issues must reach Systems Architect or CTO within 24h.

2. CORE PRODUCTS & SERVICES

2.1 Products

Product	Description
DTN Suite	Bundle Protocol (RFC 5050) store-and-forward for interplanetary communication. Nodes store data during blackouts, forward when contact windows reopen. C-based ION implementation with zero dynamic memory allocation.
Quantum-Resistant Cryptographic Core	ML-KEM, ML-DSA PQC standards securing interstellar comms, telemetry, mission data against future quantum attacks.
RhAOS	Radiation-Hardened Autonomous OS using Triple Modular Redundancy (TMR) with majority-voting logic and continuous self-scrubbing memory to correct single-event upsets from cosmic rays.
Humanity Archive Platform	OAIS-compliant multi-generational digital preservation with automated bitstream preservation, format migration, emulation for 100+ year readability. Inspired by Svalbard Seed Vault + GitHub Arctic Code Vault.

2.2 Services

- **Interstellar Software Architecture Consulting:** Hybrid quantum-classical applications for extreme environments.
- **Mission-Critical V&V:** IV&V per NASA NPR 7150.2D for safety-critical space software.
- **Deep Space AI & Automated Governance:** AI for autonomous docking, life-support, habitat management. Includes ethical alignment, bias auditing, algorithmic transparency.

2.3 Technology Stack

- **Quantum:** Qiskit, Q# for algorithms, error mitigation, cross-platform execution
 - **DTN:** C-based ION (Interplanetary Overlay Network), zero dynamic memory allocation
 - **Preservation:** Automated fixity checks, checksums, redundancy mapping; extensive metadata registries for provenance/technical constraints
-

3. SOFTWARE ARCHITECTURE & CODING STANDARDS

3.1 Approved Language Matrix

Domain	Primary	Secondary	Rationale
Flight/Embedded	C (MISRA C)	Rust	Deterministic memory, no GC, safety certification precedent
Ground Station	Python (PEP 8), C++	Go	Orchestration + performance
Quantum	Python (Qiskit), Q#	None	Standard ecosystems; interface via classical layer
Cryptographic	C, Rust	None	No managed runtimes; minimize side-channel attack surface
Data Science/ML	Python	Julia	ML ecosystem + high-performance numerical
Documentation/Config	Markdown (CommonMark), YAML, JSON	TOML	Plain-text, readable without special software for 50+ years
Build/Infrastructure	Bash, Makefile	Python	Simple, auditable, avoid framework sprawl

Absolute Runtime Rules: No dynamic memory allocation in flight software (all static/pool allocation). No exceptions in flight software (return codes only). No reflection/dynamic dispatch in safety-critical paths. All loops must have provable termination; recursion prohibited in flight. Stack depth bounded and verified.

3.2 Code Organization & Architecture

Module Design: Single responsibility per module. Explicit public interfaces only. No circular dependencies. Every module has a designated owner. Boundaries drawn at fault isolation lines (adjacent modules must operate in degraded mode when one fails).

Layered Architecture (Space Software):

Mission Application Layer (mission logic, state machines, autonomy)

- Services Layer (telemetry, command handling, comms)
- Hardware Abstraction Layer (drivers, sensor interfaces)
- OS/RTOS Layer (RtAOS or approved RTOS)

No layer may call non-adjacent layers. Mission layer contains no hardware code. HAL contains no mission logic.

State Machine Requirements: All states enumerated in module header. All valid transitions in documented state transition table. Transitions triggered only by defined events. Invalid transitions trigger FDIR subsystem.

3.3 Hybrid Quantum-Classical Architecture

Classical orchestration handles: problem decomposition/pre-processing, quantum circuit job submission, result post-processing, fallback to classical when quantum hardware unavailable or decoherence exceeds threshold. Quantum processor treated as black-box service with defined API. Quantum results validated against known bounds before mission-critical use.

3.4 Coding Conventions

Element	Convention	Example
Constants	SCREAMING_SNAKE_CASE	MAX_PACKET_SIZE
Globals	<code>g_</code> prefix, snake_case	<code>g_telemetry_buffer</code>
Functions	snake_case	<code>validate_command_sequence()</code>
Classes/Types	PascalCase	<code>BundleProtocolNode</code>
Modules	snake_case, no abbreviations	<code>radiation_hardening</code>
Booleans	<code>is_ / has_ / can_</code> prefix	<code>is_link_active</code>
Pointers (C)	<code>_ptr</code> suffix	<code>telemetry_buffer_ptr</code>

Function Design Rules: One function, one responsibility. Max 60 lines (40 for safety-critical). Max 5 arguments (use config struct beyond that). All return values checked (explicit `(void)` cast + comment for ignored). No hidden side effects — functions with side effects must be named accordingly.

Comments: Explain intent (why), not mechanics (what). Required function header: `@brief`, `@why`, `@param`, `@return`, `@note`. Inline comments only when code alone cannot communicate intent. No tautological comments.

Error Handling: All errors handled explicitly — no “can’t happen” errors in radiation environments. Error codes are named constants per module (no magic numbers). Error paths tested as thoroughly as success paths. Python: bare `except:` prohibited.

3.5 Version Control Practices

Commit Messages: Conventional Commits format: `<type>(<scope>): <short description>` then body (what changed and why, not how) then footer (issue tracker ID, breaking changes). Types: `feat`, `fix`, `docs`, `test`, `refactor`, `perf`, `ci`, `chore`.

Branch Strategy: `main` is always deployable. Feature branches: `<type>/<issue-id>-<short-description>`. Rebasing required before MR. At least one peer review + passing CI. Feature branches deleted after merge.

Merge Request Requirements: Description of what changed and why. Issue tracker reference. Evidence of unit tests for new code. Static analysis findings addressed. For safety-critical: confirmation IV&V requirements considered.

3.6 Dependency Management

Adding Dependencies: 1) Check existing approved libraries. 2) License compatibility review (AZ-LEGAL-001). 3) Vulnerability scanning — High/Critical CVE blocks adoption. 4) Engineering Lead approval. 5) Add to SBOM.

Pinning: Exact versions only (no ranges). Lock files committed to repo and updated deliberately, not automatically. Reproducible builds mandatory.

3.7 Static Analysis (CI Pipeline)

Language	Tool	Ruleset
C	PC-lint Plus / Coverity	MISRA C:2012 mandatory
Python	Pylint + Flake8	PEP 8 + project overrides
Rust	Clippy	Default + <code>clippy::pedantic</code> for safety-critical
All	SonarQube	OWASP Top 10, CWE Top 25

False positives suppressed inline with justification. Global suppression prohibited.

3.8 Code Review Checklist

Correctness: Code matches description, edge cases handled, error paths handled/tested. **Standards**

Compliance: Naming conventions, function length, return values checked, no prohibited patterns (dynamic memory, recursion, bare exceptions). **Documentation:** Function headers present/accurate, non-obvious logic explained, README updated. **Testing:** Unit tests for all new logic, error path tests included. **Security:** No hardcoded credentials, input validation at external interfaces, no unapproved dependencies.

3.9 SBOM Management

Format: SPDX (ISO/IEC 5962) v2.3+ or CycloneDX (convertible to SPDX). **Content:** Component name/version, supplier, SPDX license ID, VCS URL, SHA-256 hash, dependency relationships. Complete transitive inclusion required. **Generation:** Automated in CI pipeline. Attached to build artifacts. **Validation:** All licenses per AZ-LEGAL-001, no EOL or known-critical components, no missing transitive deps. **Usage:** Correlate with NVD/GitHub Advisories, trigger auto-rescanning on new vulns. **Archival:** Retained alongside source code/binaries; flight software SBOMs retained indefinitely. **Tools:** Python → cyclonedx-python, Rust → cargo-sbom, C/C++ → spdx-sbom-generator, Containers → syft.

3.10 Documentation Standards

ADRs: Required for major technical decisions (tech stacks, module boundaries, architectural patterns, impactful trade-offs). Filed at `/docs/adr/[NNN]-title.md` with Context, Options Considered, Decision, Consequences, Related ADRs. Status lifecycle: Proposed → Accepted → Superseded. Never deleted. Index in `/docs/adr/README.md`.

READMEs: Every repo must have README.md with: purpose, build/test instructions, run/simulation instructions, key contacts, links to ADRs/design docs/policy references.

API Documentation: All public interfaces documented in source code. External APIs additionally need machine-readable specs (OpenAPI for REST, protobuf for gRPC).

4. ENTERPRISE RISK MANAGEMENT FRAMEWORK

(AZ-RISK-001 — see also AZ-CONSOLIDATED-05 for project-level risk)

4.1 Risk Governance

Executive Board: Ultimate accountability. Approves Risk Appetite Statement, reviews Enterprise Risk Register semi-annually. **Risk Owner (COO):** Maintains Enterprise Risk Register, coordinates risk identification, escalates near-tolerance risks. **Division Risk Coordinators:** Each division designates one for domain-specific risk identification/escalation. **Risk Review Committee:** Meets quarterly (COO, CEO, CISO, Legal Officer, Engineering Lead, Finance Officer).

4.2 Risk Appetite

Low: Human life/mission safety, heritage data integrity, legal/regulatory violations (ITAR/EAR), Mission-Critical data confidentiality, government partner relationships. **Moderate:** Technology bets that may not succeed first iteration, competitive risks from tech choices, ambitious long-term R&D

financial risks. **Higher:** Calculated operational risks at space frontier, research risks in quantum/AI, principled public positions on space governance.

4.3 Enterprise Risk Taxonomy

1) **Mission & Safety** (loss of life/mission/asset) | 2) **Cyber & Info Security** (data breach, supply chain compromise) | 3) **Technology & Innovation** (tech debt, PQC standard changes, key personnel departure) | 4) **Regulatory & Legal** (ITAR, space debris, open source compliance) | 5) **Supply Chain** (foundry closure, component obsolescence) | 6) **Organizational & People** (critical personnel loss, leadership failure, skill decay) | 7) **Financial & Continuity** (contract loss, funding gap, hyperinflation) | 8) **External & Environmental** (geopolitical, solar storms, pandemic)

4.4 Risk Scoring & Response

Likelihood 1-5: Very Low (unlikely in 10y) → Very High (expected within 1y). **Impact 1-5:** Negligible → Critical (loss of life, existential). **Score = L x I.** - 1-6 (**Green**): Acceptable, monitor quarterly. - 7-14 (**Amber**): Active mitigation plan, monthly review. - 15-25 (**Red**): Immediate escalation to CEO/Board, mitigation plan within 48h. - Score 20+ or Cat 1 with Impact 4+ → immediate Executive Board escalation.

Response Strategies: Avoid, Mitigate (most common), Transfer (financial only — safety/mission cannot be transferred), Accept (low scores only, must be documented), Contingency Plan (pre-planned response).

4.5 Enterprise Risk Register

Maintained in secure document system. Fields: Risk ID, Date, Category, Description, Owner, Inherent L/I/Score, Mitigation Actions (with owners/dates), Residual L/I/Score, Response Strategy, Contingency Plan, Status, Review dates. New risks added within 5 business days. Closed risks retained permanently as learning record.

4.6 Reporting & Escalation

- Monthly: Risk Coordinator Update to COO
 - Quarterly: Risk Review Committee Report
 - Semi-Annual: Executive Risk Report to Board
 - Annual: Full Year Review + Risk Appetite reaffirmation
 - Immediate escalation to COO: New Amber/Red risks, Green→Amber/Red changes, missed mitigation due dates on Amber/Red risks, risk events realized.
-

5. EXTERNAL COMMUNICATIONS

(AZ-COMMS-001 — applies to all employees, contractors, subcontractors)

5.1 Guiding Principles

Accuracy Above Speed: Never race to be first; take time to be correct. **Consistency of Message:** All communications must reflect same understanding of mission/capabilities. **Transparency Without Recklessness:** Publish research, contribute to open standards, but respect export control, privacy, competitive confidentiality, mission safety. **One Voice:** Multiple contradictory positions damage credibility.

5.2 Designated Spokespersons

Topic	Primary	Backup
Company strategy	CEO	COO
Technology/Engineering	CTO	Engineering Lead
Security/Cybersecurity	CISO	Legal Officer
Legal/Regulatory	Legal Officer	CEO
Financial	Finance Officer	CEO
Mission Operations	COO	Mission Ops Lead
Research/Academic	CTO	Designated PI

Non-spokespeople receiving external inquiries: Thank inquirer, defer, forward to CEO immediately.

5.3 Media/Incident/Crisis Communications

Media Inquiries: All directed to CEO. Response within 4 business hours. Log all contacts in Media Contact Register. **Press Releases:** Require Technical Spokesperson → Legal Officer (IP/export control) → CEO approval chain. Issued only for significant announcements. **Incident Comms:** Default position during active incident: “We are aware, investigating, will update with confirmed information.” Speculation prohibited. Coordinated with CISO + Legal. **Crisis Comms Plan:** Maintained by CEO’s office. Crisis Comms Team (CEO, Legal, CISO, technical spokesperson) convenes within 2 hours of qualifying event. Principles: acknowledge reality, communicate known facts, commit to follow-up, protect people first, coordinate with legal.

5.4 Technical Publications & Social Media

Pre-Publication Review: Required for all conference papers, journal articles, blog posts, presentations. Minimum 10 business day review covering: export control, IP protection (patent/trade secret), classification (must be Level 1). **Conference Presentations:** Manager approval 30 days prior, pre-publication review, no Level 2+ data, standard attribution slide, no future capability commitments without CEO approval. **Social Media:** Official accounts managed by CEO's office. Personal accounts: clearly distinguish personal views from company positions, no Confidential/Mission-Critical info, no speculation on unreleased products or strategy. Professional profiles must accurately reflect title and general work area only.

5.5 Government Communications

Coordinated with Legal Officer and CEO. Includes: formal inquiries, testimony, license applications, informal briefings. Political neutrality maintained — no endorsements or donations.

6. INTELLECTUAL PROPERTY & LICENSING

(AZ-LEGAL-001)

6.1 IP Ownership

Work Made for Hire: All IP created by employees during employment (software, algorithms, documentation, designs using company time/resources/confidential info) owned by Azthreus Systems. Codified in employment contracts. **Contractor IP:** Owned by Azthreus Systems unless contract explicitly states otherwise. All contractor agreements require IP assignment clause. Contractor-retained-ownership exceptions need CTO + Legal approval. **Pre-Existing IP:** Must be disclosed in writing before incorporation. Company will not claim ownership but may require license agreement. **Joint Development:** Ownership/licensing negotiated in Joint Development Agreement before joint work begins. Legal Officer must draft/review all JDAs.

6.2 Patent Policy

Invention Disclosure: Submit Invention Disclosure Form to Legal Officer within 30 days of invention reaching documented/reduced-to-practice state. Captures: description, problem solved, prior art differentiation, all inventors, conception/reduction dates, public disclosure status. **Warning:** Public disclosure before filing patent application can permanently destroy patent rights. Consult Legal first. **Evaluation:** Legal + CTO evaluate for commercial/strategic value, patentability, cost-benefit, trade secret alternative. Decision within 60 days. **Recognition:** Inventors recognized per HR Inventor Recognition Program (ownership unaffected).

6.3 Trade Secrets

Qualifies: proprietary algorithms, quantum circuit designs, AI model parameters/configs, R&D-intensive engineering methods, non-public DTN implementation details. Protection relies entirely on maintaining secrecy: Level 3/4 classification under AZ-DATA-001, access controls, NDAs, labeling. Public disclosure permanently destroys trade secret status. Decision between patent vs. trade secret made case-by-case based on: reverse-engineering visibility, government contract disclosure requirements, useful life (>20y favors trade secret), standards body influence value.

6.4 Third-Party IP (Open Source)

License Categories: Permissive (MIT, BSD, Apache 2.0 — attribution required, permitted) | Weak Copyleft (LGPL, MPL 2.0 — modifications released under same license, permitted with legal review) | Strong Copyleft (GPL-2.0/3.0 — entire work may need GPL release, case-by-case legal + CTO approval) | Network Copyleft (AGPL-3.0 — source must be available if used over network, case-by-case) | Non-commercial (CC BY-NC, SSPL — prohibited without license negotiation) | Proprietary/No License (prohibited). **AI Training Data:** Legal must confirm license explicitly permits AI/ML training and commercial use. Web-scraped data requires specific legal review. **Copyrighted Content:** Third-party standards excerpts must be cited (not reproduced in full unless license permits). Images must be original, licensed (with documentation), or public domain. Code examples attributed with source license confirmed. **SBOM Compliance:** License info included for every component per AZ-ENG-001 Section 3.9. Legal reviews each major release SBOM.

[EXPANDED]

6.4.1 Open Source License Classification Matrix

The following matrix is the definitive reference for all open-source and third-party license evaluations at Azthreus Systems. All license determinations must reference this matrix before adoption. Disputes or edge cases escalate to Legal Officer + CTO jointly.

A. Permitted Licenses (No Special Approval Required)

License Name	SPDX ID	Version Constraints	Permitted Use Cases	Restrictions/Conditions	Approval Level
MIT License	MIT	Any version	All use cases: flight software, ground station, tools, libraries, data	Attribution notice retained in source and binary distributions	None — self-serve with SBOM entry
BSD 3-Clause	BSD-3-Clause	Any version	All use cases	No endorsement use of Azthreus name; attribution retained	None — self-serve with SBOM entry
Apache License 2.0	Apache-2.0	Any version	All use cases including patent grant	Attribution + NOTICE file preserved; patent grant conditions accepted; state changes documented	None — self-serve with SBOM entry
Python Software Foundation License	PSF-2.0	Any version	Python libraries, tools, frameworks	Attribution retained	None — self-serve with SBOM entry
Unicode License	Unicode-DFS-2016	Any version	Unicode data files and software	Attribution retained	None — self-serve with SBOM entry
Creative Commons Zero	CC0-1.0	Any version	Data, documentation, schemas (NOT software)	No attribution required; public domain dedication	None — self-serve with SBOM entry

B. Restricted Licenses (Conditional — Requires Legal + CTO Review)

License Name	SPDX ID	Version Constraints	Permitted Use Cases	Restrictions/Conditions	Approval Level
GNU LGPL v2.1	LGPL-2.1	2.1 only	Dynamic linking only in ground station tools (Python, Go, C++)	Forbidden in statically linked flight software. Must dynamically link. Modifications to the LGPL library itself must be released under LGPL-2.1. Written offer for library source code required if distributed.	Legal + CTO written approval
GNU LGPL v3.0	LGPL-3.0	3.0 only	Dynamic linking only in ground station tools	Same as LGPL-2.1 plus anti-tivoization provisions. No firmware-locked hardware.	Legal + CTO written approval
Mozilla Public License 2.0	MPL-2.0	2.0 only	File-level modifications; new files not covered	Only modified files must be released under MPL-2.0. New files may use any compatible license. Cannot be combined with incompatible copyleft in same file.	Legal review (CTO if >3 modified files)
GNU GPL v3.0	GPL-3.0	3.0 only	Internal tooling only — never for flight software, never for distributed products	Entire combined work must be GPL-3.0 if distributed. Anti-tivoization applies. For Azthreus-internal-only tools where no distribution occurs.	Legal + CTO written approval + COO sign-off if affects mission systems
GNU Affero GPL v3.0	AGPL-3.0	3.0 only	Non-networked internal services only — single-server, air-gapped, no network API exposure	Network use triggers source disclosure obligation. Forbidden for any service accessible over a network (including internal VPN).	Legal + CTO + CISO written approval

C. Prohibited Licenses (Absolute Ban)

License Name	SPDX ID	Reason for Prohibition	Alternative
GNU GPL v2.0	GPL-2.0	No patent grant; anti-tivoization absent (creates legal ambiguity for flight software); incompatible with Azthreus patent portfolio strategy	Use GPL-3.0 if copyleft required
Server Side Public License	SSPL-1.0	Copyleft triggered by providing network access; incompatible with commercial deployment models; SSPL-1.0 §13 imposes unreasonable service-level disclosure	Use Apache-2.0 or AGPL-3.0 if network copyleft is acceptable
Elastic License 2.0	ELv2	Non-compete clause prevents competing with Elastic; restrictive beyond open-source definition; blocks government contract compliance	Use OpenSearch (Apache-2.0 fork) or Apache-2.0 alternatives
Creative Commons Non-Commercial	CC-BY-NC (any version)	Non-commercial restriction incompatible with Azthreus commercial operations and government contracts	Use CC-BY or CC-BY-SA
Commons Clause	—	Adds commercial-use restriction to otherwise permissive license; violates OSD definition	Use unmodified Apache-2.0, MIT, or BSD
Server Side Public License (MongoDB)	SSPL-1.0	Same as SSPL entry above	Use Apache-2.0 alternatives
Any license with patent retaliation clause conflicting with government contracts	N/A	Patent retaliation clauses (e.g., certain MPL-1.0 variants) may conflict with ITAR/EAR obligations or government immunity provisions	Legal review case-by-case; prefer Apache-2.0 (explicit patent grant without retaliation)

D. Decision Matrix for Edge Cases

Scenario	Decision Rule
License not listed above	Submit to Legal Officer for classification before adoption. Assume Restricted until classified.
Multiple licenses in dependency tree	Most restrictive license governs. If any Prohibited license present, dependency rejected.
License version ambiguity	Legal Officer determines which version applies. If ambiguous, reject.
License text conflicts with SPDX ID	Actual license text governs. Legal Officer reconciles.

6.4.2 Dependency Chain Rule

When a **Restricted** or **Prohibited** license is introduced via a **transitive dependency** (i.e., not directly imported by Azthreus code, but pulled in by a permitted direct dependency):

1. **Immediate Flag:** Automated SBOM scanning (per §3.9) identifies the transitive dependency and raises a pipeline alert.
2. **Engineering Triage (within 5 business days):** Engineering Lead determines whether the transitive dependency can be replaced with a Permitted-license alternative. If a Permitted alternative exists, replacement is mandatory.
3. **Waiver Request (if no alternative):** If no viable Permitted alternative exists, Engineering Lead submits an Open Source License Waiver Request to Legal Officer + CTO within 5 business days. The waiver must include: - Justification for retaining the dependency - Technical analysis of why replacement is infeasible - Risk assessment (linkage type, distribution context, exposure duration) - Proposed compensating controls (e.g., isolating the dependency behind an API boundary)
4. **Decision Timeline:** Legal + CTO must approve, conditionally approve, or deny within 10 business days of waiver receipt.
5. **Conditional Approval:** May require: replacement within a defined period, isolation (dynamic linking, API boundary, process boundary), enhanced SBOM monitoring, or re-evaluation at next major version.
6. **Prohibited License Transitive Dependencies:** No waiver available. Dependency must be replaced or the parent dependency replaced. If no replacement exists for the parent dependency, escalate to Executive Board for strategic decision.

6.4.3 Dynamic vs. Static Linking Rules

Linkage Type	Flight Software	Ground Station Software	Internal Tooling
Static Linking	Only Permitted licenses (MIT, BSD-3-Clause, Apache-2.0, PSF, Unicode-DFS, CC0). GPL, LGPL, AGPL strictly forbidden — static linking triggers full copyleft obligation on flight binary.	Permitted + Restricted (with approval) licenses. GPL-3.0 forbidden if binary is distributed externally.	Permitted + Restricted (with approval). GPL-3.0 permitted if never distributed.
Dynamic Linking	Permitted licenses only (dynamic linking of GPL in flight firmware creates legal ambiguity).	Permitted + Restricted (LGPL designed for dynamic linking). GPL-3.0 forbidden if binary distributed.	Permitted + Restricted + GPL-3.0/AGPL-3.0 (if non-networked).
Process Isolation (separate address space, IPC only)	Permitted licenses. Process-isolated GPL components permitted if no code is combined.	All Permitted + Restricted. GPL components in separate process: no copyleft trigger.	All Permitted + Restricted + GPL-3.0/AGPL-3.0.
Interpretive/Scripting (Python, Lua embedded in C host)	Permitted licenses. Host does not "link" to interpreted code per copyleft definitions, but Legal must confirm for each case.	All Permitted + Restricted.	All Permitted + Restricted + GPL-3.0/AGPL-3.0.

Definition: "Static linking" = combining object files or libraries at compile/link time into a single executable image. "Dynamic linking" = loading shared libraries (`.so` , `.dll` , `.dylib`) at runtime via OS loader. "Process isolation" = separate OS process with IPC mechanism (sockets, pipes, shared memory with legal separation). "Interpretive" = code executed by an interpreter, not compiled into the host binary.

6.4.4 Source Code Distribution Obligations

For any software incorporating Restricted-license components (LGPL, MPL-2.0, GPL-3.0) that is **distributed** outside Azthreus Systems:

1. **Written Offer:** Azthreus must maintain a current written offer, valid for no less than 3 years from date of distribution, to provide complete source code for the distributed work upon request.
2. **Offer Scope:** Includes: the distributed binary, all modifications Azthreus made to the original component, and the complete source code of Azthreus's modifications plus the original component source sufficient to build the binary from source.
3. **Offer Storage:** Written offer maintained in: (a) the Humanity Archive (OAIS-compliant, indefinitely), and (b) the public-facing Azthreus legal notices page.
4. **Request Fulfillment:** Source code delivered within 10 business days of request. Delivery medium: electronic (internet archive, git repository, or physical media upon request).
5. **SBOM Traceability:** Every distribution must reference the SBOM (per §3.9) which maps each component to its license and corresponding source availability.
6. **Distribution Definition:** "Distribution" means providing the software to a third party outside Azthreus's organizational boundary, whether by download, physical media, embedded in hardware, or as a hosted service where the licensee can exercise rights to copy/modify.

6.4.5 AI-Generated Code Clause

1. **License Contamination Scan:** Any code generated by an AI coding assistant (per AZ-AI-001 §1.1) must be scanned for license contamination before inclusion in any Azthreus codebase. The scan must verify: (a) the generated code does not reproduce copyrighted material from GPL/AGPL/copyleft-licensed training data, (b) the generated code does not contain verbatim reproductions of third-party licensed code without attribution, (c) the output is sufficiently original to Azthreus's input prompt.
2. **AI Tool Commercial License Requirement:** AI coding assistants used to generate code for Azthreus projects must: (a) be covered by a commercial license that provides warranty of non-infringement and indemnification against copyright claims arising from AI-generated output, (b) the AI tool vendor must maintain training data provenance documentation sufficient to demonstrate non-infringement, (c) the AI tool must be on the CISO-approved tools list (§11.2).
3. **Output Attribution:** AI-generated code that is substantially similar to known licensed code (determined by Legal Officer review) must be attributed and treated as a third-party dependency under this §6.4.
4. **No Training Data Assumption:** The fact that an AI model was trained on open-source code does not grant Azthreus any license to the original training data or its derivatives. Each AI-generated code block is treated as a new work requiring independent license clearance.
5. **Registry Entry:** All AI-generated code integrated into production systems must be logged in the AI System Registry (AZ-AI-001 §7) with: model name/version, prompt context, license scan result, Legal Officer clearance date.

6.4.6 Zero-Day Vulnerability Response for Open Source Dependencies

1. **Monitoring:** SBOMs are continuously correlated against NVD (National Vulnerability Database), GitHub Security Advisories, and OSV (Open Source Vulnerabilities) feeds. Alerts for Critical (CVSS ≥ 9.0) and High (CVSS ≥ 7.0) CVEs trigger immediate response.
2. **Triage Timeline:** Engineering Lead triages within 24 hours of notification. Determines: affected components, affected versions, exposure in flight vs. ground vs. internal systems, exploitability assessment.
3. **Remediation SLA:** - **Critical (CVSS ≥ 9.0) in flight software:** Patch or mitigate within 72 hours. If no patch available, isolate or disable affected component. - **Critical in ground/internal:** Patch within 7 business days. - **High (CVSS 7.0-8.9):** Patch within 14 business days regardless of system type. - **Medium/Low:** Address in next scheduled sprint.
4. **Emergency Override:** If remediation SLA cannot be met, Engineering Lead + CISO jointly approve a temporary mitigation (WAF rule, network isolation, feature disable). Mitigation must have an expiration date not exceeding 30 days.
5. **Communication:** Affected system owners notified within 4 hours of confirmed Critical/High. Executive Board notified within 24 hours if flight software is affected.
6. **Post-Mortem:** Every Critical zero-day response results in a lessons-learned review within 2 weeks, feeding into improved dependency selection (§6.4.1) and monitoring configuration.

6.4.7 Future-Proofing: Emerging Technology Provisions

1. **PQC Migration Timeline:** All open-source cryptographic libraries must support ML-KEM (CRYSTALS-Kyber) and ML-DSA (CRYSTALS-Dilithium) by January 1, 2028. Libraries without PQC support are classified as Restricted and must be replaced or isolated by that date. Current PQC-supported libraries (e.g., liboqs, pqcrypto) are Permitted.
2. **AI-Generated Code Dependencies:** Dependencies identified as AI-generated (per vendor disclosure, SBOM metadata, or provenance analysis) must be flagged in the SBOM with an `AI-Generated: true` property. These dependencies receive enhanced scrutiny: license provenance must be independently verifiable, not merely asserted by the generating AI tool.
3. **Quantum Computing Libraries:** Open-source quantum computing libraries (Qiskit, Cirq, PennyLane) are Permitted for research/internal use. For flight-deployed quantum orchestration, only libraries with formal verification or formal specification are Permitted; others require Legal + CTO approval as Restricted.
4. **Post-Quantum Certificate Authority Dependencies:** Libraries implementing PQ certificate validation (e.g., liboqs-based TLS) are Permitted with priority adoption. Legacy RSA/ECDSA-only libraries must be evaluated for PQ migration path annually.

6.5 Pre-Publication Review (Details)

Required before any external publication: 1) Export Control Review (ITAR/EAR compliance). 2) IP Review (no patent-pending inventions or trade secrets without authorization). 3) Classification Review (no Level 1+ data). Applies to conference papers, journal articles, blog posts, open source

contributions, presentations, social media with technical content, press responses. Minimum 10 business day review. Do not commit to publication timelines without accounting for this.

6.6 Open Source Contributions

Approval Required If: Created during work hours, uses company equipment, contains company-derived code/knowledge, made under Azthreus Systems name. **Workflow:** Proposal (project URL, change description, target license, company-code flag) → Manager Review (no confidential info) → Legal Review (license compatibility, IP grant implications) → Pre-Publication Review → Final Approval (Manager + Legal; Engineering Lead for >500 lines or core functionality). **Rules:** Contributions under target project's license. Contributions under GPL-3.0/AGPL-3.0 require CTO approval. No unreleased code, trade secrets, Level 3/4 data, internal project names. Corporate email + copyright notice. All approved contributions logged in Open Source Contributions Register.

6.7 Releasing Internal Code as Open Source

CTO approves project selection. Remove all internal references/credentials/sensitive comments. Legal selects license (typically Apache 2.0 or MIT). CISO security review. Announce via AZ-COMMS-001 channels.

6.8 IP Violation Reporting

Infringement by Others: Report to Legal promptly. Do not contact suspected infringer directly.

Potential Violations by Azthreus Systems: Report to Legal promptly — early reporting enables pre-dispute resolution. No retaliation for good-faith reports.

[EXPANDED]

6.9 Policy Conflict Hierarchy

Where two or more Azthreus policies, standards, or procedures create conflicting obligations for the same action, data, or system, the following **mandatory precedence order** governs (highest priority first):

Priority	Domain	Authority	Examples
1 (Highest)	Export Control / ITAR / EAR	External law — no internal override permitted	ITAR 22 CFR Parts 120-130, EAR 15 CFR Parts 730-774, munitions list classification
2	Mission Safety / Life-Safety	Safety-critical requirements per NASA NPR 7150.2D and internal FDIR obligations	L4 data handling, flight software prohibitions, cognitive readiness requirements
3	Security / Data Classification	AZ-CONSOLIDATED-02	Data handling, encryption, access control, incident response
4	Legal / Intellectual Property	AZ-CONSOLIDATED-01 §6, AZ- LEGAL-001	License compliance, patent obligations, trade secret protection
5	Engineering Standards	AZ-CONSOLIDATED-01 §3	Coding standards, SBOM, static analysis, review requirements
6	AI / Ethics	AZ-CONSOLIDATED-03	AI risk tiers, EARB review, bias audit, explainability
7	Project Management	AZ-CONSOLIDATED-05	Gate reviews, sprint structure, change control
8 (Lowest)	HR / Administrative	AZ-CONSOLIDATED-04	Leave, onboarding, performance management

Application Rules: 1. When a higher-priority policy conflicts with a lower-priority policy, the higher-priority policy governs regardless of specificity. A more specific lower-priority requirement does not override a less specific higher-priority requirement. 2. When two policies at the same priority level conflict, the **more restrictive** requirement governs, unless one policy explicitly references the other with a specific exception. 3. When a policy references another policy (e.g., AZ-CONSOLIDATED-02 references AZ-ENG-001), the referencing policy governs the specific context unless the referenced policy is at a higher priority level. 4. All conflict resolutions must be documented in writing by the Legal & Compliance Officer with: the conflicting provisions identified, the priority determination, the resolution applied, and the date effective. 5. Unresolved conflicts escalate: same-priority conflicts → CTO + Legal Officer jointly; cross-priority conflicts → Executive Board. Escalation must occur within 5 business days of conflict identification. 6. This hierarchy applies within Azthreus Systems. In cases where government contract requirements or external regulations conflict with internal policies, the government/external requirement always governs. The Legal Officer must be notified immediately.